

CE307 Computer Graphics Assignment 1 - Code Explanation

Assignment: Static 2D and 3D Scenes

Course: CE307 Computer Graphics, UGIII SE

Instructor: Mustafa Mustafa, PhD

LIVE DEMO - VIEW ONLINE

CLICK HERE TO VIEW THE ASSIGNMENT ONLINE

Live Website: <https://voluble-gecko-3374dc.netlify.app/>

SOURCE CODE - GITHUB REPOSITORY

CLICK HERE TO VIEW SOURCE CODE ON GITHUB

GitHub Repository: <https://github.com/adam69iiu/graphic-design>

You can find all source code, HTML files, and project files in the GitHub repository.

Part 1: Static 2D Scene - Sierpiński Triangle

Code Structure

- 1. Canvas & WebGL Setup:** Creates 1920×1080 canvas and initializes WebGL context with antialiasing and preserved drawing buffer for PNG export.
- 2. Shaders:**
 - **Vertex Shader:** Receives position and color attributes, passes color to fragment shader
 - **Fragment Shader:** Outputs interpolated color for each pixel
- 3. Geometry Generation:** Recursive `createSierpinskiTriangle()` function subdivides initial triangle into 27 triangles across 3 levels, each with distinct colors (red, orange, yellow, green, cyan, blue).
- 4. Buffer Management:** Creates GPU buffers for vertex positions and colors, uploads data as `Float32Array`.

5. Rendering: Single `gl.drawArrays(gl.TRIANGLES, 0, count)` call renders all 27 triangles at once.

6. PNG Export: `downloadImage()` uses `canvas.toDataURL('image/png')` to save the rendered image.

Part 2: Static 3D Scene - Illuminated Pyramid

Code Structure

1. Three.js Setup: Loads Three.js library and creates Scene, PerspectiveCamera (FOV 45°, position 6,5,9), and WebGLRenderer (2560×1440, antialiasing, shadows enabled).

2. Pyramid Object: `ConeGeometry(2, 4, 4)` creates square-base pyramid with `MeshPhongMaterial` (blue color, shininess 80, specular highlights). Positioned at Y=2, casts shadows.

3. Ground Plane: `PlaneGeometry(20, 20)` rotated -90° horizontally, receives shadows.

4. Lighting System:

- **Ambient Light:** Base illumination (gray, 0.5 intensity)
- **Directional Light:** Main light source (white, 1.2 intensity, position 5,10,7.5) with 2048×2048 shadow map
- **Hemisphere Light:** Environmental lighting (sky/ground colors, 0.3 intensity)

5. Rendering: Single `renderer.render(scene, camera)` call draws the scene once.

6. PNG Export: `downloadImage()` exports canvas as PNG image.

Scene Information

- **Camera:** Perspective projection, Position (6, 5, 9), FOV 45°
- **Light:** Directional light at (5, 10, 7.5), White color, Intensity 1.2
- **Material:** Phong material, Color 0x2196F3 (Deep Blue), Shininess 80

Key Concepts

Part 1 (WebGL): Low-level GPU programming with manual shader writing, buffer management, and direct graphics pipeline control.

Part 2 (Three.js): High-level 3D graphics using built-in geometries, materials, and automatic shader generation.

Both produce high-quality static images meeting all assignment requirements: proper geometry, color blending, lighting, and single-render output.